Le stringhe

Introduzione alle stringhe in Java

Nelle lezioni precedenti abbiamo visto come in Java tutto ruoti attorno agli **oggetti**: abbiamo imparato a definire classi, creare oggetti con i costruttori, usare attributi e metodi, gestire i riferimenti e comprendere concetti fondamentali come `this`, overloading e overriding.
Abbiamo anche visto come il concetto di incapsulamento protegga lo stato degli oggetti e renda il codice più robusto, e come le classi possano appartenere a pacchetti per organizzare il codice.

Ora ci spostiamo su un argomento fondamentale per la programmazione: le **stringhe**.
In Java, le stringhe **NON sono tipi primitivi**, ma:

- **oggetti speciali della classe `String`.**
  Questo significa che, **pur comportandosi in molti casi come tipi primitivi (possiamo scriverle direttamente con i literal e concatenarle facilmente), internamente sono sequenze di caratteri immutabili**.
  Ogni stringa è quindi un **array di caratteri** nascosto, e possiamo manipolarla tramite metodi dedicati della classe `String`.

Conoscere le stringhe è essenziale perché quasi tutti i programmi lavorano con testi: nomi, messaggi, input/output, file, ecc.
Nei prossimi paragrafi vedremo come crearle, inizializzarle, concatenarle e gestire i riferimenti, prestando attenzione a concetti chiave come **immutabilità** e gestione di `null`.

Le stringhe in Java

Le stringhe in Java sono oggetti della classe `String`.
È importante ricordare che **i caratteri (`char`) non sono stringhe** e viceversa:

- **la classe `String` serve a rappresentare sequenze di caratteri, cioè testo alfanumerico**.

Internamente, una stringa è costituita da un **array di caratteri**, ma viene trattata "quasi" come un tipo primitivo, perché Java permette di usarla in modo semplice e immediato.

✓ **Vantaggi principali delle stringhe in Java:**

- **Si possono creare direttamente senza usare `new` (costruzione implicita)**.
- **La concatenazione è immediata usando l'operatore `+`**.
- **È possibile aggiungere testo a una stringa esistente con l'operatore `+=`**.

Esempi di costruzione di stringhe:

Java

```java
// Costruzione implicita tramite literal
String s = "hello";

// Costruzione tramite array di char
char[] temp = {'h','e','l','l','o'};
String stringa = new String(temp);

// Costruzione tramite costruttore
String a = new String("parola");
String b = "parola";                // equivalente assegnamento da literal

// Stringa vuota
String empty = "";

// Riferimento a null
String str = null;

// Attenzione: invocare metodi su null genera eccezione
System.out.println(str.length());  // NullPointerException
```

Nota:

anche se possiamo assegnare `null` a una stringa, **finché non viene creata un'istanza vera e propria, non è possibile invocare i metodi della classe senza generare un errore.**

Concatenazioni delle stringhe

In Java, le stringhe si possono unire tra loro usando l'**operatore** `+` oppure il metodo `concat()`.
Esempio:

Java

```java
String s = "hello";
s = s + " world";        // concatena " world" alla stringa s
// equivalente a:
s = s.concat(" world");

```

Un'alternativa più compatta è usare l'operatore `+=`:

Java

```java
String s = "hello";
s += " world";           // equivale a s = s + " world";

```

⚠ **Attenzione alla memoria:**

Anche se sembra che la stringa originale `s` venga modificata, in realtà le stringhe in Java sono **immutabili**, cioè non possono cambiare una volta create.
Quando si concatena o modifica una stringa, Java crea una nuova stringa in memoria e `s` ora punta a questa nuova zona di memoria.

Java

```java
String n = "Immutabile";
String maiuscolo = n.toUpperCase();

System.out.println(maiuscolo);  // stampa "IMMUTABILE"
System.out.println(n);          // stampa ancora "Immutabile"
```

In questo caso:

- `n` continua a puntare alla stringa originale `"Immutabile"`.
- `maiuscolo` punta a una **nuova stringa** `"IMMUTABILE"`.

Quindi tutte le operazioni sulle stringhe generano nuovi oggetti, lasciando intatto il valore originale. Questo è un concetto chiave da ricordare quando si lavora con le stringhe in Java.

Metodi di lettura

Per **leggere o estrarre informazioni da una stringa**, Java mette a disposizione alcuni metodi molto utili.
Supponiamo di avere la seguente stringa:

```java
String s = "amazing";
```

1. `length()`:
   - **Restituisce la lunghezza della stringa, cioè il numero di caratteri che contiene.**

```java
int l = s.length();  // output: 7
```

2. `charAt (int i)`:
   - **Permette di accedere al carattere che si trova in una determinata posizione della stringa.**

> **Ricorda che gli indici partono da 0, quindi il primo carattere è alla posizione 0.**

```java
char c = s.charAt(2);  // output: 'a' (il terzo carattere)
```

3. `substring(int i)`:
   - **Restituisce una sotto-stringa a partire dalla posizione indicata fino alla fine della stringa.**

```java
String ss = s.substring(3);  // output: "zing" (dalla posizione 3 in poi)
```

Metodi di ricerca nelle stringhe

Java mette a disposizione diversi metodi per **trovare la posizione di caratteri o sottostringhe** all'interno di una stringa.
Supponiamo di avere:

```java
String s = "amazing";
```

1. `indexOf(char c)`:
   - **Restituisce l'indice della prima occorrenza del carattere `c`.**

```java
int i = s.indexOf('m');  // output: 1
```

2. `indexOf(char c, int i)`:
   - **Restituisce l'indice della prima occorrenza del carattere `c`, a partire dalla posizione `i`.**

```java
int i = s.indexOf('a', 1);  // output: 2
```

3. `lastIndex(char c)`:
   - **Restituisce l'indice dell'ultima occorrenza del carattere `c`.**

```java
int i = s.lastIndexOf('a');  // output: 5
```

Questi metodi sono molto utili quando dobbiamo cercare caratteri o sottostringhe senza scorrere manualmente l'array di caratteri sottostante.

Parsing delle stringhe in numeri

Spesso in Java capita di avere stringhe che rappresentano numeri e di doverle convertire in un tipo numerico per poter fare calcoli.
Questo processo si chiama **parsing**.

Java mette a disposizione dei metodi pronti nelle classi wrapper come `Integer` e `Double`:

- `Integer.parseInt(String) :int`
  - **Converte una stringa in un intero (`int`).**

```java
String numero = "42";
int n = Integer.parseInt(numero);  // n == 42
```

> ⚠ **Se la stringa non rappresenta un numero valido, viene generata un'eccezione `NumberFormatException`.**

2. `Double.parseDouble(String): double`
   - **Converte una stringa in un numero con virgola mobile (`double`).**

```java
String valore = "3.14";
double d = Double.parseDouble(valore);  // d == 3.14
```

Anche qui, se la stringa non è un numero valido, viene sollevata una `NumberFormatException`.

Questi metodi sono fondamentali quando leggiamo input da console o file e vogliamo trattarlo come valori numerici.

Confronto fra stringhe

La classe `String` mette a disposizione metodi specifici per confrontare correttamente le stringhe.

I principali sono:

- `boolean equals(String anotherString)`
  - **Confronta il contenuto delle due stringhe.**
  - **Ritorna `true` se le stringhe hanno gli stessi caratteri nello stesso ordine, `false` altrimenti.**
- `int compareTo(String anotherString)`
  - **Confronta le stringhe in base all'ordine alfabetico (lessicografico).**

È importante distinguere questi metodi dall'operatore `==`.

Differenza tra `==` ed `equals`

L'istruzione:

```java
s == t
```

non confronta il contenuto delle stringhe, ma i **riferimenti**.

- Se `s == t` è `true`, significa che **le due variabili puntano allo stesso oggetto in memoria**
- **Due stringhe possono avere lo stesso contenuto ma riferimenti diversi**

Per questo motivo:

- `==` → **confronta le reference**
- `equals()` → **confronta il contenuto reale della stringa, carattere per carattere**

Il metodo `compareTo`

Il metodo `compareTo` restituisce un intero:

- `0` → **le due stringhe sono uguali**
- valore **> 0** → **la stringa chiamante è maggiore del parametro (`this > parametro`)**
- valore **< 0** → **la stringa chiamante è minore del parametro (`this < parametro`)**

Il confronto avviene carattere per carattere, seguendo l'ordine alfabetico (basato sui valori Unicode).

Questo metodo è molto usato per **ordinare stringhe**, ad esempio in collezioni o algoritmi di sorting.

La classe `StringBuffer`

Dopo aver visto che le stringhe in Java (`String`) sono **immutabili**, entra in gioco `StringBuffer`, che nasce proprio per risolvere questo limite.

`StringBuffer` rappresenta:

- **una sequenza di caratteri modificabile, pensata per tutti quei casi in cui una stringa deve essere costruita o alterata dinamicamente (ad esempio concatenazioni ripetute, inserimenti, modifiche progressive).**

A differenza di `String`, quindi, un oggetto `StringBuffer` **non crea una nuova istanza a ogni modifica, ma lavora sulla stessa area di memoria**, risultando più efficiente in molti scenari.

Capacità e lunghezza

Un concetto importante di `StringBuffer` è la distinzione tra:

- **lunghezza** (`length()`):
  - **numero di caratteri effettivamente contenuti**
- **capacità** (`capacity()`):
  - **spazio allocato in memoria per contenere i caratteri**

La capacità viene gestita automaticamente e può **aumentare** se necessario. Questo significa che la lunghezza non è rigidamente fissata: il buffer può espandersi quando servono più caratteri.

Costruttori principali

Vediamo ora i costruttori elencati nelle slide, chiarendone il significato pratico.

- `StringBuffer()`
  - **Crea un buffer vuoto con una capacità iniziale di 16 caratteri.**
  - **È utile quando non si conosce a priori la dimensione della stringa finale.**
- `StringBuffer(int length)`
  - **Crea un buffer vuoto con una capacità iniziale pari a `length`.**
  - Questo costruttore è utile quando si **può stimare la dimensione finale della stringa, riducendo il numero di riallocazioni in memoria.**
- `StringBuffer(String str)`
  - **Inizializza il buffer con il contenuto della stringa `str`.**
  - **La capacità iniziale sarà pari a `str.length() + 16`, lasciando spazio per eventuali append successive.**

Metodi fondamentali

`append(tipoDato var)`

Il metodo `append` aggiunge in **coda** al buffer il valore passato come parametro.

- **Accetta qualsiasi tipo di dato (int, double, char, boolean, oggetti, ecc.)**
- **Il valore viene automaticamente convertito in `String`**
- Il buffer viene modificato **in place**, senza creare nuovi oggetti

È il metodo più utilizzato per costruire stringhe dinamicamente.

```
insert(int offset, tipoDato var)
```

Il metodo `insert` **consente di inserire un valore in una posizione specifica del buffer.**

- `offset` **indica l'indice a partire dal quale avviene l'inserimento**
- **Anche in questo caso il valore viene convertito in `String`**
- I caratteri successivi vengono automaticamente spostati

Perché `StringBuffer` è importante

In sintesi, `StringBuffer` è pensato per:

- **costruzione dinamica di stringhe**
- **modifiche frequenti**
- **evitare sprechi di memoria e overhead dovuti all'immutabilità di `String`**

La classe `StringBuilder`

`StringBuilder` può essere vista come l'**evoluzione naturale di `StringBuffer`**. Nasce per lo stesso identico scopo:

- **gestire stringhe modificabili, evitando i problemi di inefficienza legati all'immutabilità di `String`.**

Dal punto di vista concettuale, quindi, `StringBuilder` e `StringBuffer` fanno la stessa cosa:

- **consentono modifiche dinamiche della stringa**
- **lavorano sullo stesso oggetto in memoria**
- **offrono praticamente gli stessi costruttori e metodi (`append`, `insert`, `delete`, `reverse`, ecc.)**

Differenza chiave: thread-safety

La differenza fondamentale, evidenziata anche nelle slide, è questa:

- `StringBuffer` **è thread-safe**
  - **i suoi metodi sono sincronizzati**
  - **più thread possono accedere allo stesso oggetto senza causare inconsistenze**
  - questa sicurezza ha un **costo in termini di prestazioni**
- `StringBuilder` **NON è thread-safe**
  - **i metodi non sono sincronizzati**
  - **non offre protezione in ambienti multithread**
  - è **più veloce** rispetto a `StringBuffer`

In altre parole, `StringBuilder` sacrifica la sicurezza in ambienti concorrenti per ottenere migliori prestazioni.

Quando usare `StringBuilder`

`StringBuilder` è la scelta consigliata nella maggior parte dei casi, in particolare quando:

- **l'applicazione è single-thread**
- **oppure l'oggetto è usato da un solo thread**
- **servono molte concatenazioni o modifiche di stringhe**

È per questo che, nello sviluppo moderno, `StringBuilder` è spesso preferito a `StringBuffer`.

Riepilogo

- `String`
  - immutabile
  - semplice e sicura
- inefficiente per modifiche frequenti
- `StringBuffer`
  - mutabile
- thread-safe
- più lento per via della sincronizzazione
- `StringBuilder`
  - mutabile

    - non thread-safe
    - più veloce
- scelta standard in contesti non concorrenti